AI/ML SYSTEM DESIGN

THE ULTIMATE BLUEPRINT CHEAT SHEET

Why are ML System Design interviews tough?

Most system design interviews aren't difficult because the architecture is complicated. They're difficult because we jump to the model before defining the system.

Inside this blueprint, we'll cover the 8-step engine:

- **Strategic HLD & LLD:** Chunking macro structures and micro APIs.
- **Visual Pipelines:** Complete data flow, serving schemas, and monitoring loops.
- **The Decision Matrix:** Master industry trade-offs with confidence.

■ KEEP THIS BLUEPRINT HANDY FOR YOUR NEXT INTERVIEW • SWIPE >>

THINK IN TWO LAYERS

How to Approach Complex Large-Scale AI Architectures

High-Level Design (HLD)

System-Wide Architecture: Focus on macro components, databases, clusters, and overall communication protocols.

- *Core Question:* How does the entire system flow together?
- *Key Output:* System block diagram, streaming vs batch pipelines, queues, and service boxes.

Low-Level Design (LLD)

Component-Level Specs: Focus on code modularity, strict schemas, data models, and resilience patterns.

- *Core Question:* How exactly do we build each component?
- *Key Output:* Request/Response JSON schemas, DB columns, class modules, backoff retries, and log structures.

■■ THE GOLDEN RULE OF SYSTEM DESIGN

Never jump to the model or algorithm in the first five minutes. A strong system designer starts with the business goals, clarifies the scale, and frames the ML problem first.

1 | START WITH THE PROBLEM

Clarify Requirements & Constraints Before Drawing Boxes

A. Business & Use Case

Define the business objective and target audience.

• *Example:* Recommend high-converting items to active users in real-time.

B. Scale & Performance

Establish the traffic and data volumes expected in production.

• *Example:* 50M DAU, average serving throughput of 5,000 QPS.

C. Core Constraints

Document strict SLAs, cost limitations, and security policies.

• *Example:* Strict latency SLA (p99 < 200ms); compliance with PII regulations.

D. Success Metrics

Connect model metrics directly to business outcomes.

• *Example:* Boost model F1-Score to lift the platform click-through rate (CTR) by 5%.

■ INTERVIEW PRO-TIP

Never skip this phase. Interviewers love candidates who ask clarifying questions about SLAs, scale, and business metrics before writing a single line of architecture.

2 | DESIGN THE DATA LAYER

From Raw Data Sources to Production Feature Stores

The Data Pipeline Chain:

1. Data Sources: Raw inputs from databases, application events, and client logs.

2. Ingestion & Validation: Streaming (Kafka) or batch ingestion. Schema validation checks for drift or anomalies.

3. Processing & Features: Transforming raw data into model features, saved in a **Feature Store** (offline for training, online for serving).

Key Data Decisions:

• **Freshness:** Real-time streaming features vs. cost-effective nightly batch features.

• **Data Quality:** Preventing target leakage and validating label alignment.

• **Governance:** PII masking, data access compliance, and feature versioning.

Pipeline Flow: Data Sources → Ingestion → Validation → Storage → Processing → Features / Embeddings

3 | CHOOSE THE AI APPROACH

Select the Simplest Solution That Solves the Problem

The Complexity Spectrum:

Start from the simplest and scale up if needed:

Classical ML (Lightweight, interpretable)
v
Deep Learning (Captures complex patterns)
v
LLM / RAG (Context-rich generation)
v
Fine-Tuning / Agents (Task specialization)

Execution Lifecycles:

Traditional ML Pipeline:

Training → Evaluation → Registry → Deployment → Inference

Generative AI Pipeline:

Prompt → Retrieval → Context → LLM → Validation

■ **Rule of Thumb:** Don't use a billion-parameter LLM when a logistic regression or XGBoost model gets you 95% of the way with 1000x less latency and cost.

4 | DESIGN SERVING

Deploying Models to Meet Production Traffic SLAs

Serving Paradigms:

• **Real-Time (Sync):** REST/gRPC endpoints for instant predictions. High scale, strict SLAs (p99 < 100ms).

• **Batch (Offline):** Cost-effective, pre-computed predictions stored in DB for instant retrieval.

• **Asynchronous (Queue):** Message queues (Kafka, Celery) for heavy processing, decoupling request from execution.

Serving Infrastructure Checklist:

• **Caching (Redis):** Bypass model inference completely for high-frequency or repeated queries.

• **Load Balancing & Autoscaling:** Distribute traffic and scale instances up/down based on CPU/GPU usage.

• **Rate Limiting:** Protect downstream models from sudden spikes and denial-of-service.

"Accuracy means little if your serving system cannot handle production traffic and maintain strict latency SLAs."

5 | EVALUATE & MONITOR

The Three Pillars of Production Health

1. Model Health

Tracks the performance and behavior of the ML model over time.

- **Accuracy & F1:** Prediction correctness.
- **Data Drift:** Change in input data distribution.
- **Concept Drift:** Change in target variable relation.
- **Bias & Fairness:** Sub-population fairness.

2. System Health

Tracks the infrastructure stability and SLA compliance.

- **Latency:** p50, p90, and p99 response times.
- **Throughput:** Requests/sec (QPS).
- **Errors:** 5xx error rates, timeout counts.
- **Resource Cost:** CPU/GPU utilization and dollars spent.

3. LLM/GenAI Quality

Tracks evaluation dimensions for LLM & RAG-based systems.

- **Groundedness:** Is the answer backed by context?
- **Relevance:** Does it answer the user prompt?
- **Hallucination:** Factuality and alignment metrics.
- **Safety:** Toxicity, PII, and prompt injection filters.

■ **Monitoring Philosophy:** Don't just log raw data. Set up automated alerts on **p99 latency spikes** and **model feature drift** to trigger automated rollbacks or retraining schedules before users feel the impact.

6 & 7 | FEEDBACK & COMPONENT LLD

From Automated Learning Flywheels to Low-Level Code Specifications

6. The Feedback Loop Flywheel

Production is not the finish line. Establish an active flywheel to capture data and continuously refine models:

User Interactions
v
Error & Failure Analysis
v
Data / Prompt Updates
v
Automated Retraining & Deploy

7. Component-Level LLD Checklist

For every single component in your architecture, you must specify the code-level details:

- **APIs & Schemas:** Strict Protobuf or JSON schemas.
- **Database Design:** Indexes, partition keys, table schemas.
- **Error Handling:** Exp-backoffs, jitter, and dead-letter queues.
- **Testing:** Unit tests, integration tests, and shadow deployments.

■ **Key Takeaway:** Designing a resilient system means accounting for failures. Always specify how your component handles API timeouts, database outages, or model failures.

8 | MASTER THE TRADE-OFFS

Architecting is About Finding the Right Balance, Not the Perfect Choice

Dimension	Option A	Option B	Decision Driver
Database	**SQL** (ACID, relational)	**NoSQL** (Flexible schema)	Complex relationships vs. horizontal scale.
Processing	**Batch** (Simple, offline)	**Streaming** (Real-time, fresh)	Compute cost vs. data freshness needs.
GenAI	**RAG** (Dynamic context)	**Fine-Tuning** (Domain style)	External knowledge vs. behavior adaptation.
Compute	**CPU** (Cost-efficient)	**GPU** (High throughput)	Inference latency SLAs vs. budget constraints.
Metrics	**Accuracy** (Heavy models)	**Latency** (Light models)	Prediction quality vs. execution speed SLAs.

■ **The Senior Architect Persona:** Never declare a single architecture as the absolute 'best.' Always present options and explain: *"We chose Option A over Option B because our requirements prioritize SLA latency over perfect accuracy."*

THE MENTAL MODEL RECAP

Your Complete End-to-End ML System Design Blueprint

The System Design Loop:

Problem → Data → Model → Serving → Evaluation → Monitoring → Feedback Loop

■ GitHub Resources

Get comprehensive study guides, interview prep sheets, and DSA code solutions.

■ CSE Insights

Practical AI/ML and software engineering content by **Simran Anand** on YouTube.

■■ Marevlo

Hands-on engineering projects and interactive tech learning tools.

■ Let's Discuss!

What is the toughest part for you?
HLD, LLD, scalability, model selection, or evaluation? Share below!

■■ Repost this if you found it useful! Save it to reference before your next interview.